

Natural Language Processing

Laboratorio #2

Representaciones básicas de texto

Instrucciones generales

- Individual.
- Entrega: Domingo 9 de agosto, 2026. 23:59.
- Trabajar en Python, en un Jupyter Notebook o script (.py).
- Este laboratorio es una continuación directa del Laboratorio #1: parta del mismo notebook/script y del mismo corpus ya cargado, limpio y tokenizado.

Antes de iniciar, asegúrese de tener disponibles en su notebook, desde el Laboratorio #1:

- El DataFrame del corpus cargado (texto original, categoría, y demás columnas relevantes).
- El pipeline de normalización completo (tokenización → minúsculas → sin puntuación → sin stopwords → lematización).
- Una columna con el texto ya normalizado por documento (lista de tokens o texto reconstruido).

Si algo de esto no quedó guardado, reconstrúyalo antes de continuar; el resto del laboratorio depende de él.

1. Bolsa de palabras (Bag of Words)

Sobre el corpus ya normalizado:

- Construya una representación de Bolsa de Palabras (BoW) para todo el corpus usando *CountVectorizer* de scikit-learn.
- Reporte la forma (shape) de la matriz resultante: número de documentos × tamaño del vocabulario.
- Explique, con sus propias palabras, qué información se pierde al representar un documento como bolsa de palabras (orden, gramática, contexto) y qué se gana a cambio.

2. Vectores de conteo y dispersión

- A partir de la matriz de conteos del punto anterior, calcule qué porcentaje de las celdas de la matriz son ceros (dispersión / sparsity).
- Grafique cómo cambia la dispersión de la matriz si se limita el vocabulario a las 100, 500, 1000 y 5000 palabras más frecuentes.

- Discuta: ¿por qué crece la dispersión al aumentar el vocabulario? ¿Qué implicaciones tiene esto en el costo de almacenamiento y de cómputo?
- Investigue y mencione brevemente qué es una matriz dispersa (sparse matrix) y por qué se usan estructuras especiales (p. ej. formato CSR) en vez de matrices densas para este tipo de datos.

3. N-gramas

Usando CountVectorizer con el parámetro `gram_range`:

- Genere una representación con unigramas ($n=1$), otra con bigramas ($n=2$).
- Para cada configuración, reporte el tamaño del vocabulario resultante.
- Liste los 15 bigramas más frecuentes del corpus normalizado y comente si aportan información que los unigramas por sí solos no capturan.
- Analice: ¿qué ventaja ofrecen los bigramas/trigramas frente a la bolsa de palabras clásica (unigramas)? ¿Qué costo tienen en tamaño de vocabulario?

4. TF-IDF

- Construya la representación TF-IDF del corpus normalizado usando `TfidfVectorizer`.
- Seleccione 3 documentos de categorías distintas y, para cada uno, muestre las 10 palabras con mayor puntaje TF-IDF. ¿Qué valor juegan estas palabras dentro de los artículos y como refleja su puntaje este papel?
- Compare esas palabras con las 10 palabras más frecuentes (por conteo simple) de los mismos documentos. ¿Coinciden? ¿Qué diferencias observa y por qué ocurren, según las fórmulas de TF e IDF vistas en clase?
- Explique con sus propias palabras por qué TF-IDF penaliza palabras muy comunes en todo el corpus y favorece palabras raras pero repetidas dentro de un documento.

5. Similitud entre documentos (similitud coseno)

- Usando los vectores TF-IDF, calcule la similitud coseno entre pares de documentos con `cosine_similarity` de `scikit-learn`.
- Seleccione un documento de referencia y encuentre los 5 documentos más similares dentro de todo el corpus. Reporte sus categorías: ¿coinciden con la categoría del documento de referencia?
- Calcule la similitud coseno promedio entre documentos de la misma categoría y compárela con la similitud coseno promedio entre documentos de categorías distintas.
- Explique, con sus propias palabras, por qué se usa similitud coseno en lugar de distancia euclidiana para comparar documentos de distinta longitud.

6. Análisis

Apoyándose en los resultados obtenidos:

- ¿Cómo cambia la dispersión de la matriz al pasar de unigramas a bigramas o trigramas? ¿Por qué?
- De las tres representaciones trabajadas (BoW, n-gramas, TF-IDF), ¿cuál le parece más adecuada para una tarea de clasificación de noticias por categoría? Justifique.
- ¿La similitud coseno agrupa razonablemente los documentos por categoría? ¿Qué categorías se confunden más entre sí y por qué cree que ocurre?
- Si tuviera que elegir una sola representación para construir un buscador de noticias similares dentro de este corpus, ¿cuál usaría y por qué?

Entregables

Entregable	Contenido
Link a repositorio Notebook o script	Código completo y comentado.
Reporte escrito (máx. 4 páginas)	Resultados de las secciones 1 a 5 y respuestas al análisis (sección 6).